

 Week 09: Retrieval-Augmented Generation (RAG)

Build it. Break it. Fix it. — From First Principles to Production Pipelines

WEEK

09

DATE

13 MAR 2026

FOCUS

RAG PIPELINES

LLMs are brilliant reasoning engines trapped inside a jar — they can only think about what they've already seen. RAG is the hand that opens the jar, reaching out to grab YOUR documents, YOUR data, and YOUR knowledge, then feeding it straight into the model's brain at query time. This week we build a complete RAG pipeline from scratch, break it with adversarial queries, and then systematically upgrade it from naive to production-grade.

 Quick Links & Resources

Resource	Link
 Lecture Slides	Google Drive
 Colab Notebook	Google Colab (Local Copy: Codes/cClass7.ipynb)
 Excalidraw Board	Interactive Whiteboard
 Assignment	Google Colab — Last Cell

 Table of Contents

#	Section	Core Question
	The Core Problem	Why can't LLMs just answer from their training data?
1	RAG Workflow & When to Use It	RAG vs. Fine-Tuning vs. Context Stuffing — which do I pick?
2	Ingestion 1: Document Parsing	Why does "garbage in, garbage out" matter so much?
3	Ingestion 2: Chunking Strategies	Fixed vs. Recursive — and why 80% of RAG systems fail here.
4	Contextual Retrieval	How do you stop chunks from losing their identity?
5	Vector Storage & Embeddings	Embeddings are the brain. Vector DBs are the memory.
6	Retrieval 1: Hybrid Search	Why does semantic search alone always fail on dates and IDs?
7	Retrieval 2: Reranking & Compression	The Two-Pass System: Fast & Rough → Slow & Precise.
8	Query Transformation	Rewriting, Multi-Query, HyDE, and Step-Back Prompting.
9	RAG Prompt Design & Full Pipeline	Grounding, citations, and the end-to-end lifecycle.

#	Section	Core Question
	Hands-On: Building RAG from Scratch	6 progressive exercises from the Colab notebook.
	RAG Metrics: Precision vs. Recall	How do we measure retrieval quality?
	Vector DB Architecture Deep Dive	Collections, Payloads, HNSW, and multimodal storage.
	The Excalidraw Masterclass	U-Shaped Attention, the Job Analogy, and system design.
	The Upgrade Path & Homework	From Level 0 to Level 7 — each level fixes a specific failure.

The Core Problem: Why LLMs Need RAG

The LLM Paradox: Large Language Models are extraordinary reasoning engines, but they are fundamentally *isolated*. They live inside a frozen snapshot of the internet, unable to see your world.

Problem	What It Means	Real-World Example
 Frozen in Time	Models only know their training data. Knowledge cutoff = blindness beyond that date.	"What were ACME's Q3 2024 results?" → Model has no idea if trained before Oct 2024.
 No Private Knowledge	LLMs trained on public internet. They've never seen your internal docs.	"What's our remote work stipend?" → Model will hallucinate a generic answer.
 Confident Hallucination	When they don't know, they don't say "I don't know" — they <i>invent</i> answers with full confidence.	"What JIRA ticket tracks the Redis replication bug?" → Model fabricates a ticket number.

[!CAUTION] "It only knows its training data. Your docs don't exist in its world." Without RAG, your LLM is just a very expensive, very confident guesser about your private data.

1 The RAG Workflow & When To Use It

The RAG Pipeline in One Line: User Question → **RETRIEVE** relevant chunks → **AUGMENT** the prompt with them → **GENERATE** a grounded answer

When to Use What

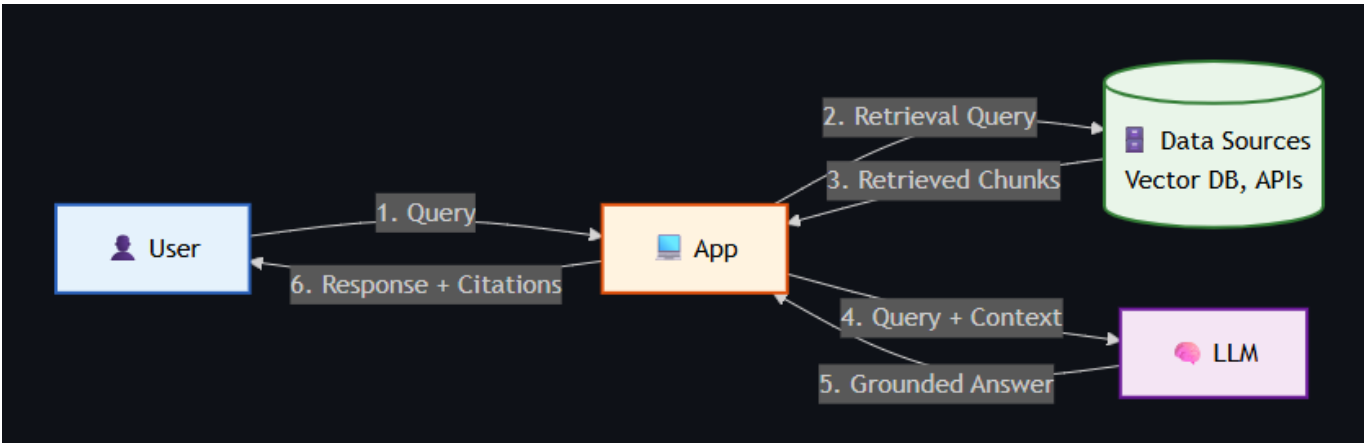
Strategy	When to Use	When NOT to Use
Context Window Stuffing	Small knowledge base (< 500 pages). Just dump it all in the prompt.	Dataset is huge (10K+ pages). Prompt cost explodes.
Fine-Tuning	Model needs a new <i>skill, style, or format</i> .	Adding factual knowledge. Fine-tuning for facts = unreliable.

Strategy	When to Use	When NOT to Use
RAG	Model doesn't <i>know</i> something. External/private data. Too large to fit in a single prompt.	When you need the model to change its <i>behavior</i> , not its knowledge.

[!TIP] **The Decision Rule:**

- Want the model to **know something new**? → **RAG**
- Want the model to **do something differently**? → **Fine-tuning**
- Information is small enough to fit in the prompt? → **Context Stuffing** (but if data changes, RAG is easier to maintain)

The End-to-End Architecture



2 Ingestion Part 1: Document Loading & Parsing

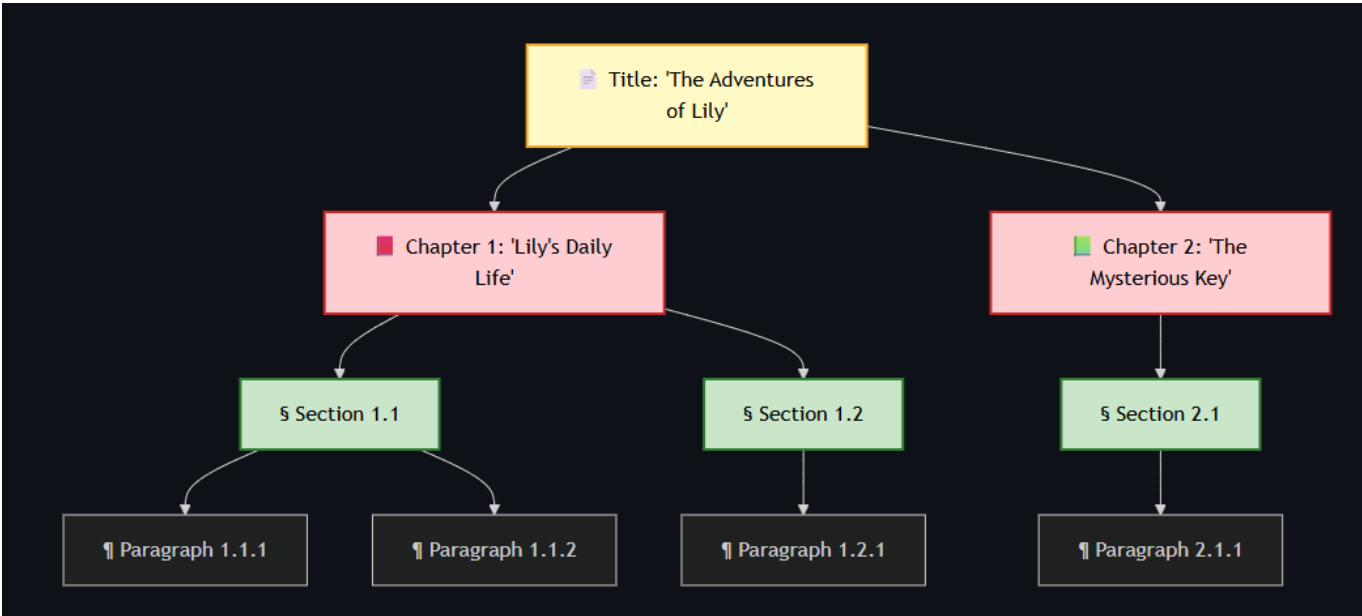
[!WARNING] **GARBAGE IN, GARBAGE OUT.** *"If your parser can't handle structure, your RAG is broken before it starts."*

Raw documents are almost never clean text. The quality of your parser directly determines the ceiling of your entire RAG system.

Format	The Parsing Nightmare	What Breaks
PDFs	Multi-column layouts read left-to-right ignoring columns.	Tables get completely destroyed or misaligned.
Word (.docx)	Structural headers and metadata are lost as raw text.	Hierarchical structure (H1 > H2 > H3) is flattened.
HTML	Navigation bars, footers, ads, and <code><script></code> tags dilute content.	80% of extracted text is noise, not content.

The Ideal: Structure-Aware Parsing

A good parser preserves the document's **tree structure** — chapters → sections → paragraphs — so that chunking can happen at logical boundaries, not arbitrary character counts.



Document Parsing Toolbox

Tool	Best For	Key Advantage
Unstructured.io	General purpose, multi-format	Standardizes various file types into one JSON schema.
LlamaParse	Complex PDFs, nested tables	Uses vision models to salvage layouts other parsers destroy.
Docling	High-speed, structure-aware	Open-source, fast, deep understanding of document hierarchy.

3 Ingestion Part 2: Chunking Strategies

Once parsed, a document is too large to embed as a single vector. It must be chopped into smaller pieces ("chunks"). **This is where 80% of RAG systems fail.**

[!CAUTION] "Every cut destroys context. The art of chunking is choosing where to cut."

Strategy 1: Fixed-Size Chunking (The Naive Approach)

How: Split every N tokens with overlap. Ignore all boundaries. **Problem:** Often cuts mid-sentence, mid-paragraph, or mid-thought. **Faithfulness Score:** ~0.47 💔

```
def chunk_fixed(text: str, chunk_size: int = 200, overlap: int = 50) -> List[str]:
    """Split text into fixed-size word chunks with overlap."""
    words = text.split()
    chunks = []
    for i in range(0, len(words), chunk_size - overlap):
        chunk = " ".join(words[i:i + chunk_size])
        if chunk.strip():
            chunks.append(chunk.strip())
    return chunks
```

Strategy 2: Recursive Chunking (The Standard)

How: Attempts to split logically: paragraphs → sentences → characters. Respects the natural boundaries of human writing. **Faithfulness Score:** ~0.80 ☒

```
def chunk_recursive(text: str, max_size: int = 200) -> List[str]:
    """Split text on natural boundaries: paragraphs, then sentences."""
    paragraphs = [p.strip() for p in text.split("\n\n") if p.strip()]
    chunks = []
    for para in paragraphs:
        words = para.split()
        if len(words) <= max_size:
            chunks.append(para)
        else:
            sentences = para.replace(". ", ".\n").split("\n")
            current, current_len = [], 0
            for sent in sentences:
                sent_len = len(sent.split())
                if current_len + sent_len > max_size and current:
                    chunks.append(" ".join(current))
                    current, current_len = [sent], sent_len
                else:
                    current.append(sent)
                    current_len += sent_len
            if current:
                chunks.append(" ".join(current))
    return [c for c in chunks if len(c.split()) > 10]
```

Head-to-Head Comparison (from Notebook)

```
=====
FIXED-SIZE CHUNKING (chunk_size=100, overlap=20)
=====
Chunk 1 (100 words):
    Q3 2024 Financial Results – ACME Corporation Revenue and Growth ACME Corporation
    reported total revenue of $4.2 billion for Q3 2024, representing a 12 ...
    ⚠ Cuts mid-thought: revenue section runs into employee growth!

=====
RECURSIVE CHUNKING (max_size=100)
=====
Chunk 1 (61 words):
    Revenue and Growth
    ACME Corporation reported total revenue of $4.2 billion for Q3 2024,
    representing a 12% increase year-over-year...
    ☒ Clean paragraph boundary. Topic: Revenue only.

Chunk 2 (44 words):
```

Employee Growth
The company added 1,847 new employees during Q3, bringing total headcount to 34,500...
☒ Separate topic. Clean boundary.

 **Fixed cuts mid-thought. Recursive respects paragraph boundaries.** Always prefer recursive unless you have a specific reason for fixed-size.

Strategy 3: Advanced Methods

Method	How It Works	Best For
Structure-aware	Chunks on Markdown headers (#, ##, ###).	Docs with clear header hierarchy.
AST-based	Parses code via Abstract Syntax Trees. Functions/classes stay intact.	Code repositories.
Parent-Child (Small-to-Big)	Embed <i>small</i> chunks for precision search; retrieve the <i>larger parent</i> chunk for context.	Best of both worlds.

4 Contextual Retrieval: The Anthropic Approach

The Problem: Lost Context

Imagine a chunk says: "revenue grew 3%". But **which company? Which year? Which quarter?** The chunk was ripped from its document — its identity is lost.

The Fix: LLM-Powered Context Injection

Use an LLM during the **ingestion phase** (one-time cost) to inject surrounding context into every chunk *before* embedding it.

```
def contextualize_chunk(chunk: str, full_doc: str, title: str) -> str:
    """Prepend LLM-generated context to a chunk before embedding."""
    resp = ant.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=150,
        messages=[{"role": "user", "content": f"""<document title="{title}">
{full_doc}
</document>

Here is a chunk from that document:
<chunk>
{chunk}
</chunk>

Write a SHORT (2-3 sentence) context that situates this chunk within the document.
Include: which document, what section/topic, key entities or time periods.
This will be prepended to the chunk for search.
```

```
Context:""]}]
)
ctx = resp.content[0].text.strip()
return f"{ctx}\n\n{chunk}"
```

Before vs. After

BEFORE contextualization:
"Revenue and Growth
ACME Corporation reported total revenue of \$4.2 billion for Q3 2024..."

AFTER contextualization:
"This excerpt is from ACME Corporation's Q3 2024 financial report, specifically from the 'Revenue and Growth' section. It provides a breakdown of the company's \$4.2 billion quarterly revenue across three main business divisions: cloud services, hardware, and consulting services.

Revenue and Growth
ACME Corporation reported total revenue of \$4.2 billion for Q3 2024..."

[!IMPORTANT] **Result: 67% fewer retrieval failures** when combined with BM25 and reranking. It's a one-time ingestion cost for a massive query-time gain. The embedding now captures *what the chunk is about*, not just what it literally says.

5 Vector Storage & Embeddings

Embeddings: The Brain 🧠

Embeddings transform text into high-dimensional vectors (arrays of floats, e.g., 1536 dimensions). **Similar meanings = nearby points in vector space.**

The word "puppy" and the phrase "young dog" will have vectors pointing in almost the same direction, even though they share zero characters.

Top Embedding Models:

Provider	Model	Dimensions	Best For
OpenAI	text-embedding-3-small	1536	General purpose, great quality/cost ratio
Cohere	embed-english-v3	1024	Multi-language, strong performance
Jina AI	jina-embeddings-v3	1024	Open-source alternative

Vector Databases: The Memory 📁

You can't do a SQL `SELECT * WHERE text LIKE '%puppy%'` for *concepts*. You need **Approximate Nearest Neighbor (ANN)** search to find the closest vectors in milliseconds.

Tier	Tools	Use Case
Prototyping	ChromaDB, FAISS	Local development, fast iteration. Run in-memory.
Production	Pinecone, Weaviate, Qdrant	Scalable, cloud-native, billion-vector capable.
SQL-Native	pgvector (PostgreSQL)	Already have Postgres? Add vector search as an extension.

[!IMPORTANT] **Metadata is Crucial!** Never store just the vector. **Always** attach metadata:

```
{ "source": "handbook_remote.pdf", "title": "Remote Work Policy", "page": 3,
  "date": "2024-01-15" }
```

This is essential for **filtering** (pre-search) and **citations** (post-generation).

6 Retrieval Part 1: Hybrid Search & Filtering

Relying *only* on vector search is a classic mistake. Industry consensus: **Hybrid beats semantic alone. Every time.**

The Two Search Paradigms

Search Type	Mechanism	Strength	Weakness
Semantic (Dense)	Embedding cosine similarity. Finds <i>meaning</i> .	"car" matches "automobile". Great for natural language.	Terrible at exact matches: dates, IDs, commands.
Keyword (BM25 / Sparse)	Token frequency (TF-IDF). Finds <i>exact words</i> .	"JIRA AUTH-2847" found instantly. Dates, codes, IDs.	Fails on synonyms: "sick leave" vs "medical absence".

Reciprocal Rank Fusion (RRF): The Merging Algorithm

RRF fuses two ranked lists into one. It's elegant — **no hyperparameters to tune.**

```
def reciprocal_rank_fusion(semantic, keyword, k=60):
    """Merge two ranked lists with RRF. Simple, effective, no tuning needed."""
    scores = {}
    for rank, (idx, _) in enumerate(semantic):
        scores[idx] = scores.get(idx, 0) + 1 / (k + rank + 1)
    for rank, (idx, _) in enumerate(keyword):
        scores[idx] = scores.get(idx, 0) + 1 / (k + rank + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```


Metadata Filtering: The "WHERE" Clause

"If you know the user is asking about Q3, why search Q1 and Q2? Filter first, then search."

```
# Filter by date, source, or category BEFORE vector search
query = "Revenue docs FROM 2024"
results = vector_db.search(query, filter={"year": 2024})
```

Pre-filtering trims the search space entirely — faster **and** more accurate.

7 Retrieval Part 2: Reranking & Compression

The Two-Pass Retrieval System

One search isn't enough for high quality. You need **two passes**:



Pass	Model Type	Speed	What It Does
Pass 1 (Bi-Encoder)	Embeds query and docs separately.	⚡ Fast	Scans millions of records. Returns top 50-100 candidates.
Pass 2 (Cross-Encoder)	Reads [Query + Document] together.	🐢 Slow	Understands deep nuance. Re-orders top 50 → final best 3-5.

Implementation:

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(question: str, results: List[Dict], top_k: int = 3) -> List[Dict]:
    """Rerank results with cross-encoder."""
    pairs = [(question, r["chunk"]) for r in results]
    scores = reranker.predict(pairs)
    for r, s in zip(results, scores):
        r["rerank_score"] = float(s)
    return sorted(results, key=lambda x: x["rerank_score"], reverse=True)[:top_k]
```

Contextual Compression

Even highly relevant chunks are often **90% noise and 10% gold**. All that noise dilutes the LLM's limited attention window.

Solution: Use a fast, cheap LLM to extract *only* the relevant sentences from a chunk before inserting it into the final prompt.

"The highest-ROI upgrade after chunking. Gets more important as your data grows."

8 Query Transformation: Making Questions Better

Users ask terrible questions. They say things like "What does it mean?" where "it" relies on chat history context. Query transformation fixes the user's raw input before it hits the vector database.

Strategy	How It Works	When to Use
1. Rewriting	LLM rewrites messy chat history into a clean, standalone search query.	Chatbot follow-up questions (anaphora resolution).
2. Multi-Query	Generate 3-5 different phrasings of the same question. Search all. Deduplicate results.	When a single phrasing might miss relevant docs.
3. HyDE	Generate a <i>fake answer</i> first. Embed the fake answer and search with it.	<i>"The hypothetical answer is often closer to matching documents than the original short question."</i>
4. Step-Back	Ask a broader background question first, retrieve foundational context, then answer the specific question.	Complex questions requiring prerequisite knowledge.

9 RAG Prompt Design & The Full Pipeline

[!CAUTION] "RAG without citations is just a fancier hallucination machine."

The 4 Rules of Good RAG Prompts

Rule	Instruction to the LLM	Why
1. Grounding	"Answer <i>ONLY</i> from the provided context."	Stops the model from using parametric knowledge (its training data).
2. Escape Hatch	"If you don't know, say 'I don't have enough information.'"	Prevents confident hallucinations when the context doesn't contain the answer.
3. Citations	"Cite your sources with metadata."	Receipts. Users can verify. Trust is built on traceability.
4. Structure	Label each chunk clearly using <i>[Source: doc_title]</i> tags.	Helps the model distinguish between multiple retrieved chunks.

The Actual Prompt Template (from Notebook)

```
prompt = f"""Answer based ONLY on the context below.
If the context doesn't have the answer, say "I don't have enough information."
```

```
Cite your sources.
```

```
Context:  
{context}
```

```
Question: {question}
```

```
Answer: ""
```

The True End-to-End Pipeline

INGESTION PHASE (*Async, run once or on data update*):

```
Docs → Parse → Chunk (recursive) → Contextualize (LLM) → Embed → Store in Vector  
DB + BM25 Index
```

RETRIEVAL & GENERATION PHASE (*Real-time, per query*):

```
User Question → Transform Query → Hybrid Search (Semantic + BM25 → RRF) → Rerank  
(Cross-Encoder) → Compress → Augment Prompt → LLM → Answer with Citations
```

Hands-On: Building RAG from Scratch (Notebook Walkthrough)

The Colab notebook ([class7.ipynb](#)) builds a complete RAG pipeline in **6 progressive exercises** against 17 fictional ACME Corp documents. Here's the full walkthrough:

Setup

```
!pip install -q openai chromadb rank_bm25 sentence-transformers pypdf tiktoken  
anthropic  
  
from openai import OpenAI  
import chromadb  
  
oai = OpenAI() # Uses OPENAI_API_KEY  
  
def get_embeddings(texts, model="text-embedding-3-small"):   
    """Batch embed texts with OpenAI."""   
    cleaned = [t.replace("\n", " ").strip() for t in texts]   
    resp = oai.embeddings.create(input=cleaned, model=model)   
    return [d.embedding for d in resp.data]
```

Exercise 1: Chunking (Why It Matters)

Compares Fixed-Size vs. Recursive chunking on the same ACME Q3 Report. **Result:** Fixed cuts mid-thought; Recursive respects paragraph boundaries.

Exercise 2: Naive RAG (Build It, Then Break It)

The simplest pipeline: `chunk` → `embed` → `store in ChromaDB` → `semantic search` → `generate`.

```
def naive_rag(question, k=5):
    """Simplest RAG: semantic search → stuff prompt → generate."""
    results = collection.query(query_embeddings=[get_embedding(question)],
n_results=k)
    context = "\n".join(f"[Source: {m['title']}] \n {d}" for d, m in
zip(results["documents"][0], results["metadatas"][0]))

    resp = oai.chat.completions.create(
        model="gpt-4o-mini", temperature=0,
        messages=[{"role": "user", "content": f"""Answer based ONLY on the context
below.
If the context doesn't have the answer, say "I don't have enough information."
Cite your sources.

Context:
{context}

Question: {question}

Answer: """}]}
    )
    return resp.choices[0].message.content
```

☑ Queries that work:

- 🔍 "What was ACME's total revenue in Q3 2024?"
 💬 ACME's total revenue in Q3 2024 was \$4.2 billion. [Source: Company Q3 2024 Report]
- 🔍 "What is the home office stipend for remote workers?"
 💬 The home office stipend for remote workers is \$1,500. [Source: Employee Handbook – Remote Work Policy]

★ Queries that expose weaknesses:

- 🔍 "What changed on 2024-09-15?"
 → Semantic search finds the right chunk (dist=0.551), but it's barely confident.
 → Embeddings are TERRIBLE with dates – "2024-09-15" is just noise to a semantic model.

🔍 "What is the Slack command to trigger a rollback?"
 → Semantic search finds the right doc (dist=0.428), but by luck – the slash command
 "/deploy rollback" is a string match problem, not a meaning problem.

Exercise 3: Hybrid Search (Semantic + BM25)

BM25 catches what embeddings miss: **exact dates, slash commands, JIRA IDs, acronyms.**

```
from rank_bm25 import BM25Okapi

bm25 = BM25Okapi([tokenize(c) for c in all_chunks])

def hybrid_search(question, k=5):
    """Combine semantic + BM25 with RRF."""
    sem = collection.query(query_embeddings=[get_embedding(question)],
n_results=20)
    sem_ranked = [(int(id.split("_")[1]), dist) for id, dist in zip(sem["ids"][0],
sem["distances"][0])]

    scores = bm25.get_scores(tokenize(question))
    bm25_ranked = sorted(enumerate(scores), key=lambda x: x[1], reverse=True)[:20]

    fused = reciprocal_rank_fusion(sem_ranked, bm25_ranked)
    return [{"chunk": all_chunks[idx], "meta": chunk_meta[idx], "score": sc} for
idx, sc in fused[:k]]
```

Results: The same date query ("What changed on 2024-09-15?") now has the Authentication System chunk at #1 via hybrid search, because BM25 matched "2024-09-15" as a literal string.

Exercise 4: Contextual Retrieval (Anthropic's Technique)

Before embedding, use Claude to prepend context to every chunk. The embedding now captures *which document, which section, which time period*.

Result: Cross-document queries like "What is ACME's AI strategy and how does it connect to current products?" now correctly retrieve Board Meeting Notes + Q3 Product Launches together.

Exercise 5: Reranking (The Precision Upgrade)

After hybrid search returns top 10, a Cross-Encoder re-scores them by reading [Query + Chunk] together:

🔍 "What are the known issues with ACME's authentication system?"

Top 3 after reranking:

- [1] rerank=9.027 | Engineering Wiki – Authentication System (Known Issues) ←
- 👉 Perfect
- [2] rerank=2.053 | Engineering Wiki – Authentication System (Overview)
- [3] rerank=1.947 | Engineering Wiki – Authentication System (TokenService)

- 💬 The known issues are:
- 1. Token blacklist in Redis not replicated across regions (AUTH-2847)
 - 2. SAML integration with Okta has intermittent timeouts (AUTH-3021)
 - 3. Magic link flow missing separate rate limiting (AUTH-3156)

Exercise 6: Side-by-Side Comparison

Same complex query through all three approaches:

Query: "What is the Slack rollback command and what are the security requirements for remote workers?"

Approach	Result Quality	Why
🕸 Naive RAG	✅ (lucky)	Semantic search barely finds both chunks (dist=0.512, 0.515). Fragile.
🔍 Hybrid Search	✅ (reliable)	BM25 catches "rollback" and "remote" as exact keywords. Both at RRF=0.0325.
🔗 Full Pipeline	✅ (precise)	Contextual chunks + hybrid + reranker. Top chunks are exactly the right sections. Most confident.

[!NOTE] **Watch the retrieved chunks, not just the answer.** The LLM might paper over bad retrieval with its parametric knowledge, but the chunks reveal the true quality of your pipeline.

📊 Deep Dive: RAG Metrics (Precision vs. Recall)

When evaluating a retrieval system, we look at a **Confusion Matrix** of chunks:

	Retrieved ✅	Not Retrieved ❌
Relevant	True Positive (TP) 🎯	False Negative (FN) 🚫
Irrelevant	False Positive (FP) 🗑️	True Negative (TN) ✅

The Two Key Metrics

Metric	Formula	Asks	Trade-off
Precision	$\frac{TP}{TP + FP}$	"Of everything I retrieved, how much was useful?"	High precision = little noise, but might miss things.
Recall	$\frac{TP}{TP + FN}$	"Of all relevant chunks in the DB, how many did I find?"	High recall = you got all the gold, but also tons of garbage.

In RAG, optimize for Recall first (so the LLM always has the answer somewhere in context), then use **Reranking** to boost Precision (filter out the garbage before it hits the prompt).

Deep Dive: Vector Database Architecture

A Vector Database is far more than an array of embeddings. It's a distributed system engineered for sub-millisecond similarity search at scale.

Core Components

Component	Role
AI Services	The embedding model (external). Turns text → vectors.
Clients / SDKs	Python, Rust, Go, TypeScript SDKs for developer interaction.
Query Planner + HTTP/gRPC Proxy	Routes requests, handles batching and load balancing.
Vector Indexes (HNSW)	The heart. Uses HNSW (Hierarchical Navigable Small World) graphs — no linear $O(N)$ scans.
Background Workers	Continuously re-index and optimize as new data streams in.

How Data Is Stored (Qdrant Model)

Collections contain **Points**. Each Point has:

- `idx` — Unique ID
- `vector` — The `[0.12, -0.45, ...]` array representing meaning
- `Payload` — The metadata (e.g., `{Species: "Dog", Color: "Brown", URL: "site.com/img.jpg"}`)

Distance Metrics:

Metric	Formula	When to Use
Cosine Similarity	$\cos(\theta) = A \cdot B / (\ A\ \cdot \ B\)$	Normalized embeddings (most common).
Dot Product	$A \cdot B$	When magnitude matters (e.g., popularity signals).
Euclidean Distance	$\ A - B\ _2$	Spatial clustering tasks.

The Excalidraw Masterclass

The whiteboard session distilled the RAG paradigm into core visual principles.

Why You Can't Just "Stuff the Context"

Imagine a 1000-page book. You *could* try to embed the whole thing as one vector, but you'd be forcing the model to compress an impossibly rich document into a single fixed-size array. The signal-to-noise ratio is completely destroyed.

Analogy: It's like summarizing an entire human life in one word. You lose all nuance. You **must** break it into logical parts.

The U-Shaped Attention Curve (Lost in the Middle)

LLMs don't pay equal attention to all parts of a long prompt. Their attention is **U-shaped**:

- **Strong** at the very **beginning** of the prompt
- **Strong** at the very **end** of the prompt
- **Weak** in the **middle** — the "Lost in the Middle" syndrome

The Fix: After reranking, place the most important chunks at the **end** of the context (right before the question), not buried in the middle.

The Job Application Analogy (Two-Pass Retrieval)

The clearest way to understand Bi-Encoders vs. Cross-Encoders:

1. **10,000 people apply for a job.** (*All chunks in your DB*)
2. **Pass 1: Bi-Encoder (The Resume Scanner)**
 - A fast, automated system scans resumes for keywords: "5 years experience", "AI skills", "Python".
 - **Result:** 10,000 → 100 candidates. Fast, but rough.
3. **Pass 2: Cross-Encoder (The Human Interview)**
 - You don't hire all 100 off a keyword scan. You now sit down and compare each candidate *against the specific job description*, reading both together.
 - **Result:** 100 → Top 5 hires. Slow, but precise.

The Three Pillars of RAG

Pillar	Steps	Key Insight
Ingestion	Parse → Chunk → Contextualize → Embed → Store	If parsing fails, everything downstream is garbage.
Retrieval	Transform Query → Hybrid Search → Rerank → Compress	Two passes: recall-first (cast wide net), then precision (interview the candidates).
Generation	Augment Prompt → LLM → Answer with Citations	Beware the U-shaped attention curve. Place gold at the end.

🔧 The Upgrade Path & Homework

Each level fixes a **specific failure**. Start simple. Upgrade where it hurts.

Level 0: Stuff context window	(small docs only)
Level 1: Naive semantic search	← We started here (Exercise 2)
Level 2: Better chunking	(recursive, structure-aware – Exercise 1)
Level 3: Contextual retrieval	(Anthropic's technique – Exercise 4)
Level 4: Hybrid search	(semantic + BM25 + RRF – Exercise 3)
Level 5: Reranking	(cross-encoder – Exercise 5)

Level 6: Query transformation (next class)
Level 7: Agentic RAG (next class)

Homework Missions

1. **BUILD:** Replace the sample docs with YOUR own data (lecture notes, a textbook, project docs). Run the same tests.
2. **BREAK:** Find YOUR failure cases — what queries break your pipeline? Exact terms? Dates? Cross-document synthesis?
3. **COMPARE:** Try two chunking strategies on your data. Which works better? Why?
4. **READ:** [Anthropic's Contextual Retrieval Blog Post](#)
5. **THINK:** Where did your pipeline fail? Which specific upgrade level would fix it?

Next class: Advanced RAG patterns (Self-RAG, GraphRAG, RAPTOR, Agentic RAG) and the fascinating story of *why Claude Code abandoned RAG entirely*.

Made with ❤️ by [Akshat Shethia](#)